



ЛР 3. Gitlab CI/CD

▼ Задача

Создать и настроить свой первый пайплайн в Gitlab

▼ Гайд

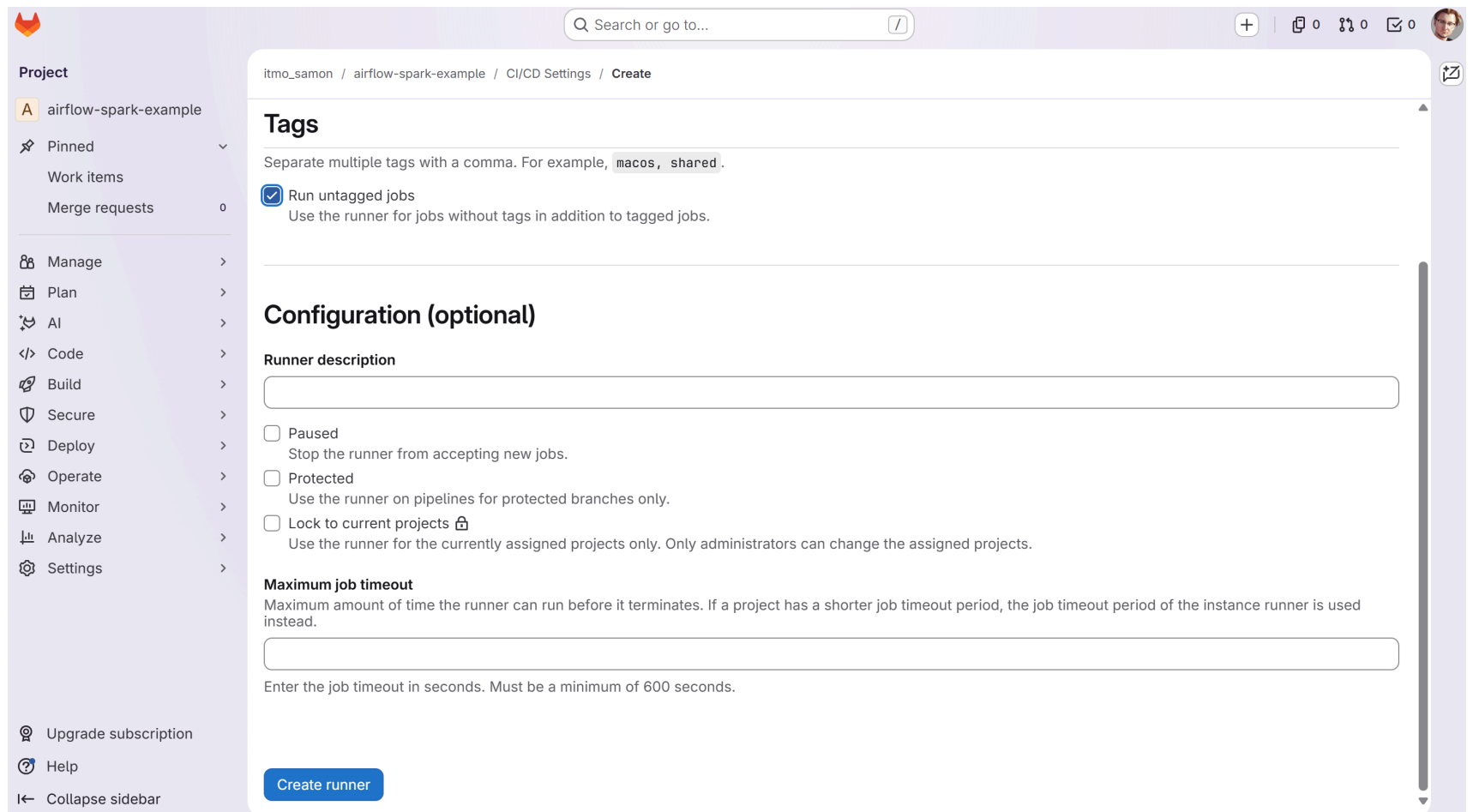
▼ Часть1. Установка и настройка Gitlab

1. На сервере, где будет целевой деплой (можно локально) создать файл `docker-compose.yml` для локального деплоя раннера

▼ `docker-compose.yml`

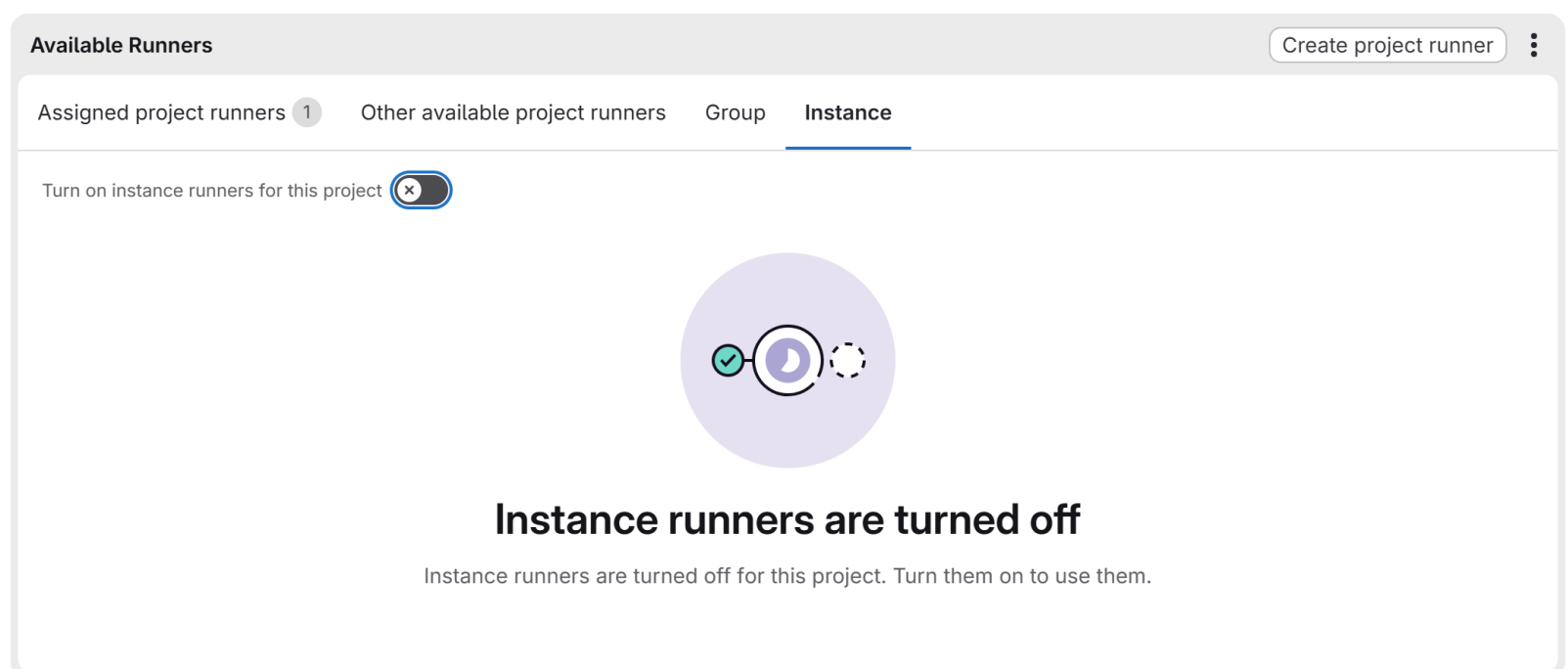
```
services:
  gitlab-runner:
    image: gitlab/gitlab-runner:alpine
    container_name: gitlab-runner
    restart: always
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
```

2. В терминале выполнить `docker-compose up -d`, подождать пока деплой завершится (проверить командой `docker ps`)
3. В Gitlab в желаемом проекте перейти в раздел *Settings - CI/CD*, там раскрыть пункт *Runners* и создать новый project runner (например). Разрешить запуск нетегированных джоб, если используется gitlab.com - не забыть отключить instance runners (=shared)



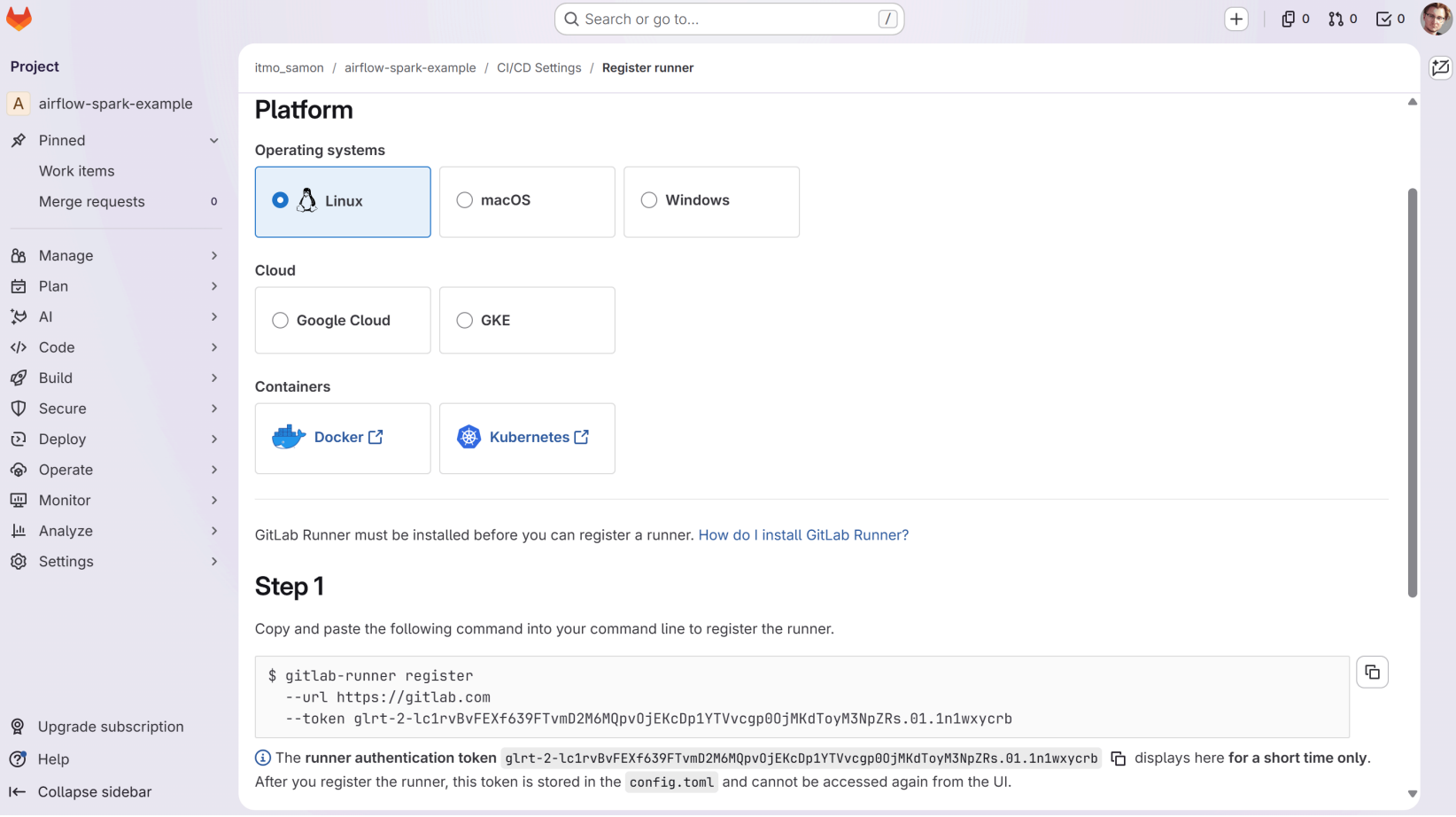
Runners

Runners are processes that pick up and execute CI/CD jobs for GitLab. [What is GitLab Runner?](#)



4. Скопировать команду для регистрации раннера и прогнать (через docker exec) там же, где развернут локальный раннер (адрес сервера может отличаться)

▼ Пример:



```
llidd@lenid MINGW64 ~/itmo
$ docker-compose up -d
[+] up 2/2
✓ Network itmo_default Created 0.0s
✓ Container gitlab-runner Started 0.2s

llidd@lenid MINGW64 ~/itmo
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS   NAMES
02be5e2007a4   gitlab/gitlab-runner:alpine         "/usr/bin/dumb-init ..." 4 seconds ago  Up 3 seconds        gitlab-runner

llidd@lenid MINGW64 ~/itmo
$ docker exec -it gitlab-runner gitlab-runner register --url https://gitlab.com --token glrt-2-lc1rvBvFEXf639FTvmD2M6MQpv0jEKcDp1YTVvcgp00jMKdToyM3NpZRs.01.1n1wxycrb
Runtime platform arch=amd64 os=linux pid=19 revision=5265d41d version=18.11.1
Running in system-mode.

Enter the GitLab instance URL (for example, https://gitlab.com/):
[https://gitlab.com]:
Verifying runner... is valid correlation_id=9f31070a1ec28474-LED runner=2-lc1rvBv runner_name=02be5e2007a4
Enter a name for the runner. This is stored only in the local config.toml file:
[02be5e2007a4]: test-runner
Enter an executor: docker-autoscaler, docker+machine, kubernetes, virtualbox, shell, custom, instance, docker-windows, ssh, parallels, docker:
docker
Enter the default Docker image (for example, ruby:3.3):
docker:latest
Runner registered successfully. Feel free to start it, but if it's running already the config should be automatically reloaded!

Configuration (with the authentication token) was saved in "/etc/gitlab-runner/config.toml"

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug gitlab-runner
  Learn more at https://docs.docker.com/go/debug-cli/

llidd@lenid MINGW64 ~/itmo
$
```

5. После первичной инициализации раннера, необходимо его донстроить: залогиниться внутрь контейнера через `docker exec -it gitlab-runner bash` и открыть файл `/etc/gitlab-runner/config.toml`

▼ В файле дописать/отредактировать строчку:

```
...
volumes = ["/var/run/docker.sock:/var/run/docker.sock", "/cache"]
...
```

6. Вернуться в веб-интерфейс Gitlab и убедиться, что раннер “подцепился” (в том же разделе Runners)

▼ Пример:

▼ **Runners**

Runners are processes that pick up and execute CI/CD jobs for GitLab. [What is GitLab Runner?](#)

Available Runners			Create project runner	
Assigned project runners 1			Other available project runners 1	Group Instance 123
Status	Runner configuration ?	Owner ?		
Online Idle	#52902523 (2-1c1rvB) Project 0 Last contact: 5 minutes ago 31.134.140.54 Created by l1dd 8 minutes ago	airflow-spark-example		

▼ **Часть 2. Создание пайплайн**

- 1. Перейти в свой Gitlab-репозиторий
- 4. В корне репозитория создать конфиг самого пайплайна

▼ `.gitlab-ci.yml`

```
stages:
  - build # стейдж для билда образа
  - deploy # стейдж для деплоя сервиса

build-job:
  image: docker:latest # чтоб собрать докер образ, нам нужен.. докер
  stage: build
  script:
    - ls -la # убедимся что раннер видит содержимое репозитория
    - docker build -t image_name . # image_name = атрибут image в docker-compose.
yml

deploy-job:
  stage: deploy
  image: tmaier/docker-compose:latest # чтоб сделать деплой через композ нам нуже
н.. КОМПОЗ
  script:
    - echo "Deploying application..."
    - ls -la
    - docker-compose up -d # деплоим по-старинке
    - echo "Application successfully deployed!"

clear-job: # дополнительная джоба, которая одноразово сотрет задеплоенное
  stage: deploy
  image: tmaier/docker-compose:latest
  script:
    - docker-compose rm -sf
  when: manual # только ручное выполнение (после выполнения лабораторной, наприме
p)
```

- 5. После коммита файла `.gitlab-ci.yml` пайплайн запустится автоматически, ход выполнения можно посмотреть в разделе *CI/CD - Pipelines* (может занять несколько минут)

▼ Как это должно выглядеть:



Search or go to...

Project

A

airflow-spark-example

Pinned

Issues

Merge requests

Manage

Plan

Code

Build

Pipelines

Jobs

Pipeline editor

Pipeline schedules

Artifacts

itmo_samon > airflow-spark-example > Pipelines > #1030435287

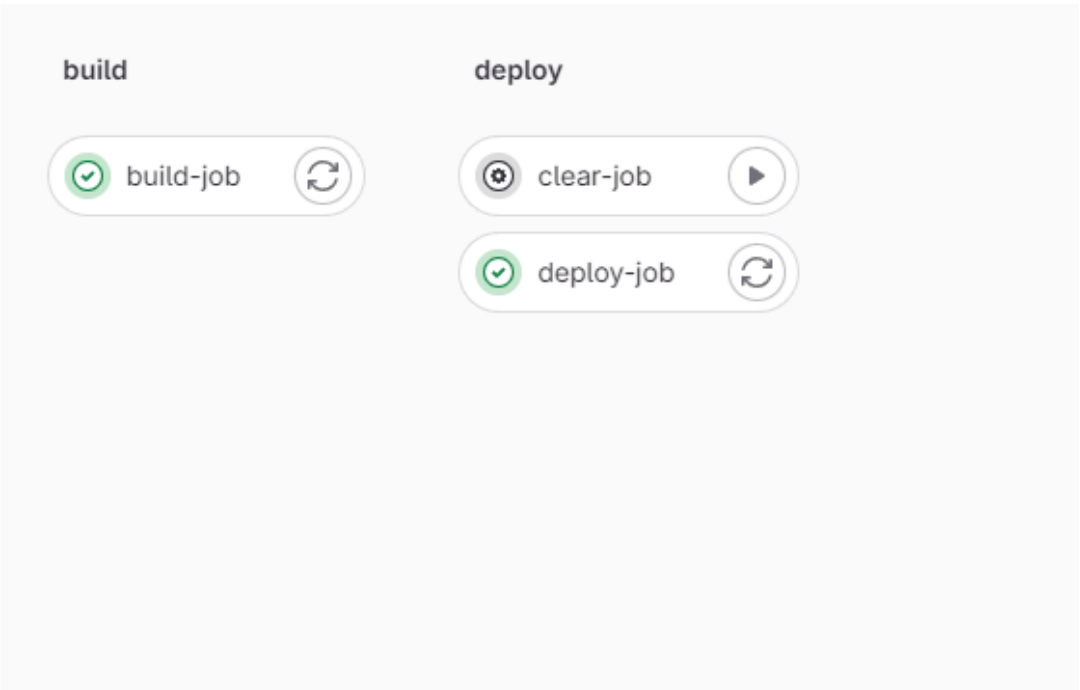
Update .gitlab-ci.yml

Passed llidd created pipeline for commit [6e7a8d51](#) finished just now

For [cicd](#)

latest 3 Jobs 0 18 seconds, queued for 1 seconds

Pipeline Needs Jobs 3 Tests 0



itmo_samon > airflow-spark-example > Jobs > #5252235833

build-job

Passed Started 1 minute ago by llidd

Search job log

```
1 Running with gitlab-runner 16.4.1 (d89a789a)
2   on example_runner yPkJfW3y, system ID: r_devqsTUVbiyK
3   Preparing the "docker" executor
4   Using Docker executor with image docker:latest ...
5   Pulling docker image docker:latest ...
6   Using docker image sha256:ca9c28c1e8a77b55b236a2d7f49864cf23c4f83c8726bb028435492c2c25cb7b for docker:latest with digest docker@sha256:f28ffd78641197871fea8fd679f2bf8a1cdafa4dc3f1ce3e708ad964aac2879a ...
7   Preparing environment
8   Running on runner-ypkjfw3y-project-58862411-concurrent-0 via 32adfe06da98...
9   Getting source from Git repository
10  Fetching changes with git depth set to 20...
11  Reinitialized existing Git repository in /builds/itmo_samon/airflow-spark-example/.git/
12  Checking out 6e7a8d51 as detached HEAD (ref is cicd)...
13  Skipping Git submodules setup
14  Executing "step_script" stage of the job script
15  Using docker image sha256:ca9c28c1e8a77b55b236a2d7f49864cf23c4f83c8726bb028435492c2c25cb7b for docker:latest with digest docker@sha256:f28ffd78641197871fea8fd679f2bf8a1cdafa4dc3f1ce3e708ad964aac2879a ...
16  $ ls -la
17  total 40
18  drwxr-xr-x  5 root   root   4096 Oct  9 12:33 .
19  drwxr-xr-x  4 root   root   4096 Oct  9 12:28 ..
20  drwxr-xr-x  6 root   root   4096 Oct  9 12:52 .git
21  -rw-r--r--  1 root   root    6 Oct  9 12:28 .gitignore
22  -rw-rw-rw-  1 root   root   698 Oct  9 12:33 .gitlab-ci.yml
23  -rw-r--r--  1 root   root   227 Oct  9 12:28 Dockerfile
24  -rw-r--r--  1 root   root    71 Oct  9 12:28 README.md
25  drwxr-xr-x  2 root   root   4096 Oct  9 12:28 dags
26  -rw-r--r--  1 root   root  1555 Oct  9 12:28 docker-compose.yml
27  drwxr-xr-x  2 root   root   4096 Oct  9 12:28 spark
28  $ docker build -t my_airflow_image .
```

deploy-job

✓ Passed Started 1 minute ago by  ilidd

```
1 Running with gitlab-runner 16.4.1 (d89a789a)
2   on example_runner yPkJfW3y, system ID: r_devqsTUVbiyK
3   ✓ Preparing the "docker" executor
4     Using Docker executor with image tmaier/docker-compose:latest ...
5     Pulling docker image tmaier/docker-compose:latest ...
6     Using docker image sha256:aa43178c58fa94d49791f28d41c771431ddcff6585b5e2c10b083e24d3258954 for tmaier/docker-compose:latest with digest tmaier/docker-compose@sha256:c1baf6aa05b698e89b37e1
7   ✓ Preparing environment
8     Running on runner-ypkjfw3y-project-50862411-concurrent-0 via 32adfe06da98...
9   ✓ Getting source from Git repository
10    Fetching changes with git depth set to 20...
11    Reinitialized existing Git repository in /builds/itmo_samon/airflow-spark-example/.git/
12    Checking out 6e7a8d51 as detached HEAD (ref is c1cd)...
13    Skipping Git submodules setup
14  ✓ Executing "step_script" stage of the job script
15    Using docker image sha256:aa43178c58fa94d49791f28d41c771431ddcff6585b5e2c10b083e24d3258954 for tmaier/docker-compose:latest with digest tmaier/docker-compose@sha256:c1baf6aa05b698e89b37e1
16    $ echo "Deploying application..."
17    Deploying application...
18    $ ls -la
19    total 40
20    drwxr-xr-x  5 root  root    4096 Oct  9 12:33 .
21    drwxr-xr-x  4 root  root    4096 Oct  9 12:20 ..
22    drwxr-xr-x  6 root  root    4096 Oct  9 12:52 .git
23    -rw-r--r--  1 root  root      6 Oct  9 12:20 .gitignore
24    -rw-rw-rw-  1 root  root    698 Oct  9 12:33 .gitlab-ci.yml
25    -rw-r--r--  1 root  root    227 Oct  9 12:20 Dockerfile
26    -rw-r--r--  1 root  root     71 Oct  9 12:20 README.md
27    drwxr-xr-x  2 root  root    4096 Oct  9 12:20 dags
28    -rw-r--r--  1 root  root   1555 Oct  9 12:20 docker-compose.yml
29    drwxr-xr-x  2 root  root    4096 Oct  9 12:20 spark
30    $ docker-compose rm -sf
31    No stopped containers
32    $ docker-compose up -d
```

6. Если шаг 5 пройден и все окрашено “зеленым”, значит пайплайн успешно отработал, т.е. сбилдил приложение и задеплоил его. В терминале можно убедиться, что тестовое приложение успешно развернулось (например с помощью команды `docker ps`)

▼ Задание

Необходимо сделать следующее:

- Добавить стейдж `test` и соответствующую джобу до этапа билда, выполняющую “тест” - можно сделать что-нибудь простое, например проверить что директории *dags/* и *spark/* существуют и не потерялись. Главное чтоб этот синтетический тест обрабатывал и не давал пайплайну выполняться дальше, если джоба упала
- Добавить правило, чтоб пайплайн осуществлял шаг деплоя автоматически только для веток *main*, *master* и *develop*
- Шаг билда не должен выполняться автоматически, если ветка начинается с префикса *feature/*
- Шаг тестирования выполняется всегда во всех ветках
- Добавить в настройки раннера тег и заставить шаги пайплайна выполняться только на этом тегированном раннере

▼ Отчетность

Ссылка на Gitlab-репозиторий от команды, где содержится

- файл .gitlab-ci.yml
- успешно выполненный пайплайн(ы)
- файл CHANGES.md (если еще не), где отражены изменения, сделанные в рамках этой лабораторной работы (по сравнению с 1-ой и 2-ой)